

INFORME TÉCNICO INTEGRADO

Proyecto de Inteligencia de Negocios – SUTRAN (Perú)

1. Introducción técnica y alcance del informe

El presente Informe Técnico Integrado documenta y justifica el diseño e implementación de una arquitectura de Inteligencia de Negocios (BI) en Google Cloud Platform (GCP).

El proyecto toma como caso de estudio los datos abiertos de siniestros de tránsito administrados por SUTRAN / ONSV, correspondientes al período 2018–2023, los cuales presentan características típicas de escenarios de Big Data: alto volumen, heterogeneidad de fuentes, problemas de calidad de datos y necesidad de análisis multidimensional para la toma de decisiones.

El alcance del informe comprende:

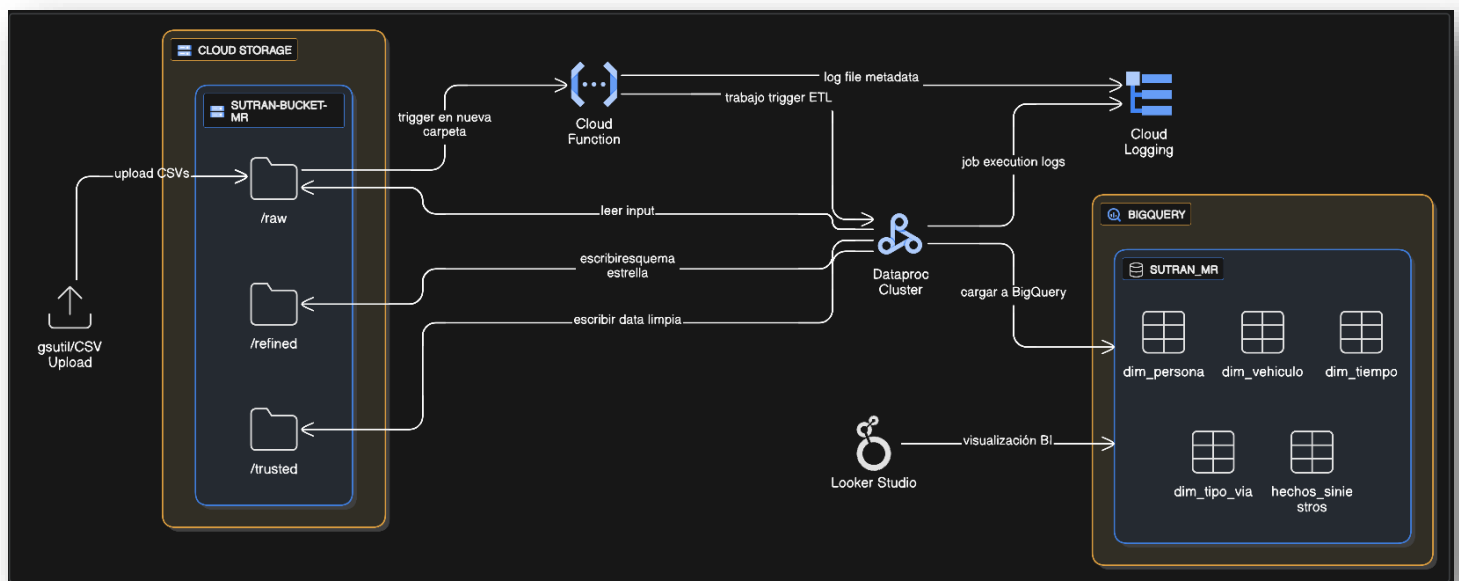
- La definición y justificación de la arquitectura Cloud adoptada.
- El uso e integración de servicios GCP para almacenamiento, procesamiento, analítica y visualización.
- El diseño del Data Lake bajo el enfoque Medallion (Bronce–Plata–Oro).
- La implementación de un modelo dimensional tipo estrella.
- El desarrollo de procesos ETL automatizados utilizando PySpark y Dataproc.
- La validación analítica mediante consultas SQL avanzadas en BigQuery.
- La implementación de visualizaciones finales para análisis de siniestros.
- El uso del repositorio GitHub como mecanismo de control de versiones y evidencia del desarrollo.

2. Arquitectura Cloud definida

2.1 Enfoque general de la arquitectura

La arquitectura implementada sigue un enfoque Data Lake → ETL → Data Warehouse → BI, desplegada íntegramente sobre Google Cloud Platform (GCP).

- Desacoplamiento de capas (storage, procesamiento, analítica).
- Procesamiento distribuido batch para grandes volúmenes de datos.
- Analítica serverless para optimizar costos y rendimiento.
- Seguridad por diseño, aplicando el principio de mínimo privilegio.
- Automatización basada en eventos para la ejecución del ETL.



2.2 Componentes principales de la arquitectura

La arquitectura Cloud implementada se compone de los siguientes componentes:

a) Capa de almacenamiento – Data Lake (Cloud Storage)

Google Cloud Storage se utiliza como repositorio central de datos bajo el patrón Medallion, permitiendo:

- Persistencia de datos originales sin modificar.
- Trazabilidad completa del proceso ETL.
- Reprocesamiento controlado ante cambios en la lógica de negocio.

El Data Lake se encuentra desplegado en un bucket multirregión (US) para mejorar la disponibilidad y tolerancia a fallos.

b) Capa de procesamiento – ETL distribuido (Dataproc + PySpark)

El procesamiento batch se realiza mediante Dataproc, utilizando Apache Spark (PySpark) para:

- Limpieza de datos (encoding, valores nulos, duplicados).
- Normalización y transformación de atributos.
- Construcción del modelo dimensional.
- Escritura de resultados en capas intermedias y finales.

Dataproc fue seleccionado por su integración nativa con GCP, escalabilidad horizontal y compatibilidad con PySpark, lo que permite manejar grandes volúmenes de datos de forma eficiente.

c) Automatización del ETL – Arquitectura orientada a eventos

Para la automatización del proceso ETL, se implementó una arquitectura event-driven, donde:

1. Un archivo CSV es cargado en la carpeta /raw del Data Lake.
2. Se activa automáticamente una Cloud Function.
3. La función dispara un job de PySpark en Dataproc.
4. El procesamiento genera datos limpios en /trusted y datos modelados en /refined.

d) Capa analítica – Data Warehouse (BigQuery)

Google BigQuery se utiliza como Data Warehouse analítico, donde se aloja el modelo estrella del proyecto. BigQuery fue elegido por:

- Su modelo serverless, sin necesidad de administrar infraestructura.
- Alto rendimiento para consultas analíticas complejas.
- Integración directa con herramientas de visualización.
- Soporte nativo para SQL analítico avanzado.

Las tablas de hechos y dimensiones son cargadas desde la capa /refined del Data Lake.

e) Capa de visualización – BI Cloud (Looker Studio)

La capa de consumo se implementa con Looker Studio, conectado directamente a BigQuery, permitiendo:

- Visualización interactiva de KPIs.
- Exploración multidimensional de siniestros.
- Validación en tiempo casi real de cambios en los datos.

2.3 Consideraciones de escalabilidad, disponibilidad y seguridad

- **Escalabilidad:** Dataproc permite escalar nodos de procesamiento y BigQuery escala de forma automática según la carga.
- **Alta disponibilidad:** uso de buckets multirregión y datasets BigQuery en ubicación US.
- **Seguridad:**
 - IAM granular por usuario y servicio.
 - VPC dedicada con subredes pública y privada.
 - Reglas de firewall restrictivas.
 - Auditoría activa mediante Cloud Logging y Audit Logs.

3. Diseño del Data Lake (Bronce – Plata – Oro)

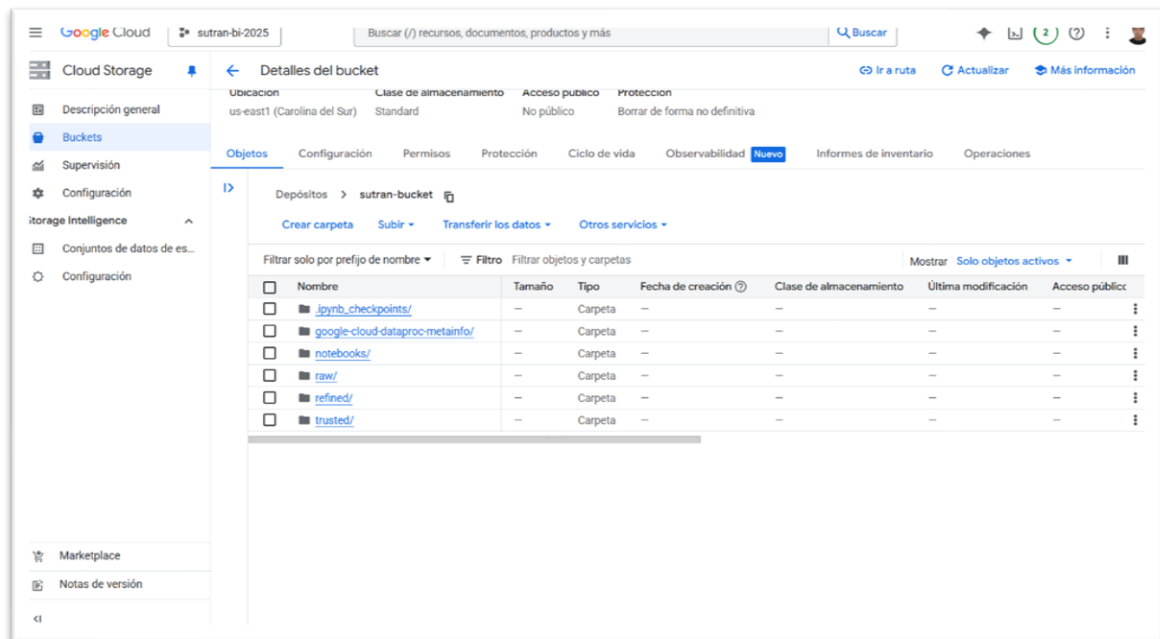
El Data Lake del proyecto se implementó en Google Cloud Storage bajo el enfoque Medallion Architecture, separando claramente las responsabilidades de ingestión, limpieza y modelado de datos. Esta decisión permite trazabilidad, control de calidad y reprocesamiento sin afectar las capas finales.

3.1 Estructura física del Data Lake

El bucket principal sutran-bucket-mr se organizó de la siguiente manera:

sutran-bucket-mr/

- └─ raw/ → datos originales sin transformar
- └─ trusted/ → datos limpios y estandarizados
- └─ refined/ → datos modelados (esquema estrella)
- └─ notebooks/ → notebooks ETL (PySpark)
- └─ scripts/ → scripts ETL productivos



- Vista del bucket mostrando /raw, /trusted, /refined
(captura del navegador GCP – imagen 1)

3.2 Capa Bronce (raw)

La capa raw contiene los archivos CSV originales descargados de fuentes abiertas de SUTRAN/ONSV.

Características técnicas:

- Formato original CSV.
- Sin modificaciones estructurales.
- Fuente única de verdad para reprocesos.
- Carga mediante gsutil.

`gsutil cp *.csv gs://sutran-bucket-mr/raw/`

```
Administrador: Windows PowerShell
PS C:\WINDOWS\system32> gsutil cp "C:\Users\jairo\Documents\datasets_sutran\BBDD_ONSV-PERSONAS_2021-2023.csv" gs://sutran-bucket-mr/raw/
Copying file://C:\Users\jairo\Documents\datasets_sutran\BBDD_ONSV-PERSONAS_2021-2023.csv [Content-Type=application/vnd.ms-excel]...
| [1 files][976.7 KiB/976.7 KiB]
Operation completed over 1 objects/976.7 KiB.
PS C:\WINDOWS\system32>
```

Creación del Cluster

...

```
gcloud functions deploy etl_trigger_sutran \
  --runtime python310 \
  --trigger-resource sutran-bucket-mr \
  --trigger-event google.storage.object.finalize \
```

```
--entry-point trigger_dataproc_job \
--source . \
--region us-central1 \
--no-gen2 \
--set-env-vars "PROJECT_ID=shaped-icon-478404-p0,REGION=us-east1,CLUSTER=cluster-sutran"
'''
```

3.3 Capa Plata (trusted)

La capa trusted almacena los datos limpios y estandarizados, resultado del primer proceso ETL en PySpark.

Transformaciones aplicadas:

- Corrección de codificación (ISO-8859-1 → UTF-8).
- Eliminación de caracteres BOM (\uffff).
- Conversión de tipos de datos.
- Eliminación de registros nulos y duplicados.
- Escritura en formato Parquet para eficiencia.

4. LIMPIEZA COMÚN Y FORMATEO

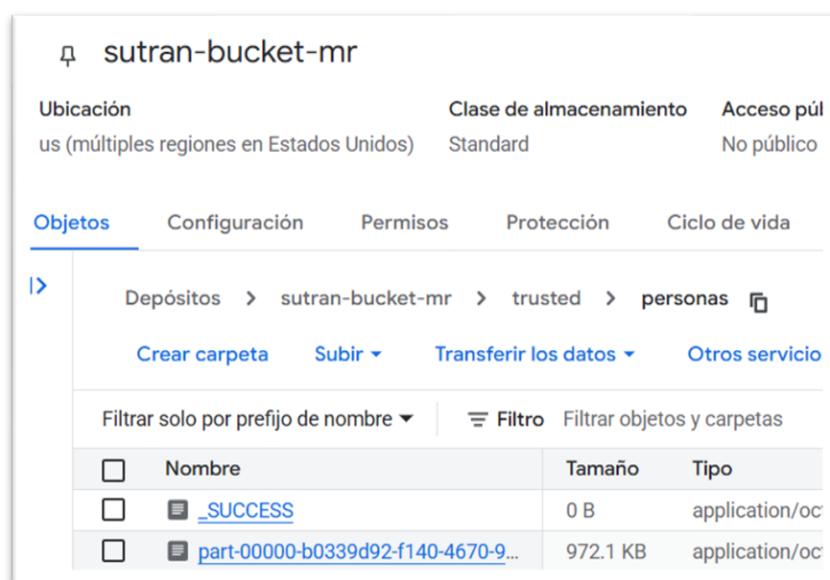
```
def limpiar_columnas(df):
    for c in df.columns:
        df = df.withColumnRenamed(c, c.strip().lower().replace(" ", "_").replace("á", "a").replace("é", "e")
                                   .replace("í", "i").replace("ó", "o").replace("ú", "u")
                                   .replace("ñ", "n"))

    return df

df_personas_clean = limpiar_columnas(df_personas_clean)
df_vehiculos_clean = limpiar_columnas(df_vehiculos_clean)
df_siniestros_clean = limpiar_columnas(df_siniestros_clean)
```

Limpieza_Inicial_Datasets_ONSV.ipynb

(ruta: /notebooks/)



Archivos Parquet generados en /trusted.

3.4 Capa Oro (refined)

La capa refined contiene los datos listos para análisis, estructurados bajo un modelo dimensional tipo estrella.

Características:

- Datos consolidados y normalizados.
- Separación de hechos y dimensiones.
- Estructura alineada a BigQuery.
- Fuente directa para carga analítica.

3. DIMENSIONES

3.1 DIM TIEMPO

```
df_fechas = df_personas.select(to_date(col("fecha"), "dd/MM/yyyy").alias("fecha")).dropna().distinct()
df_dim_tiempo = df_fechas.withColumn("id_tiempo", date_format("fecha", "yyyyMMdd").cast("int")) \
    .withColumn("anio", year("fecha")) \
    .withColumn("mes", month("fecha")) \
    .withColumn("dia", dayofmonth("fecha")) \
    .withColumn("dia_semana", date_format("fecha", "EEEE")) \
    .withColumn("trimestre", quarter("fecha")) \
    .withColumn("es_fin_de_semana", when(col("dia_semana").isin("Saturday", "Sunday"), lit(True)).otherwise(lit(False)))
```

3.2 DIM PERSONA

```
df_dim_persona = df_personas.select(
    col("codigo_persona").alias("id_persona"),
    col("tipo_persona"),
    col("sexo"),
    col("edad").cast("int"),
    col("gravedad"),
    col("situacion_de_persona"),
    col("estado_licencia"),
    col("clase_licencia"),
    col("resultado_del_dosaje_etilico_cualitativo")
).dropDuplicates(["id_persona"])
```

3.3 DIM VEHICULO

```
: df_dim_vehiculo = df_vehiculos.select(
    col("codigo_vehiculo").alias("id_vehiculo"),
    col("situacion_vehiculo"),
    col("modalidad_de_transporte"),
    col("estado_soat"),
    col("tipo_seguro"),
    col("compania_seguro"),
    col("posee_citv"),
    col("estado_citv"),
    col("lugar_impacto_vehiculo"),
    col("vehiculo") # tipo de vehículo
).dropDuplicates(["id_vehiculo"])
```

3.4 DIM TIPO_VIA

```
: vias_p = df_personas.select("tipo_de_via")
vias_v = df_vehiculos.select("tipo_de_via")
vias_s = df_siniestros.select("tipo_de_via")

dim_tipo_via = vias_p.union(vias_v).union(vias_s).distinct() \
    .withColumn("tipo_de_via_normalizado", lower(trim(col("tipo_de_via")))) \
    .filter(col("tipo_de_via_normalizado").isNotNull()) \
    .dropDuplicates(["tipo_de_via_normalizado"]) \
    .withColumn("id_tipo_via", monotonically_increasing_id())
```


4. HECHOS SINIESTROS

Base: personas

```
] df_hechos = df_personas.select(
    col("codigo_siniestro"),
    col("codigo_persona"),
    col("codigo_vehiculo"),
    to_date(col("fecha"), "dd/MM/yyyy").alias("fecha"),
    lower(trim(col("tipo_de_via"))).alias("tipo_de_via_normalizado")
)
```

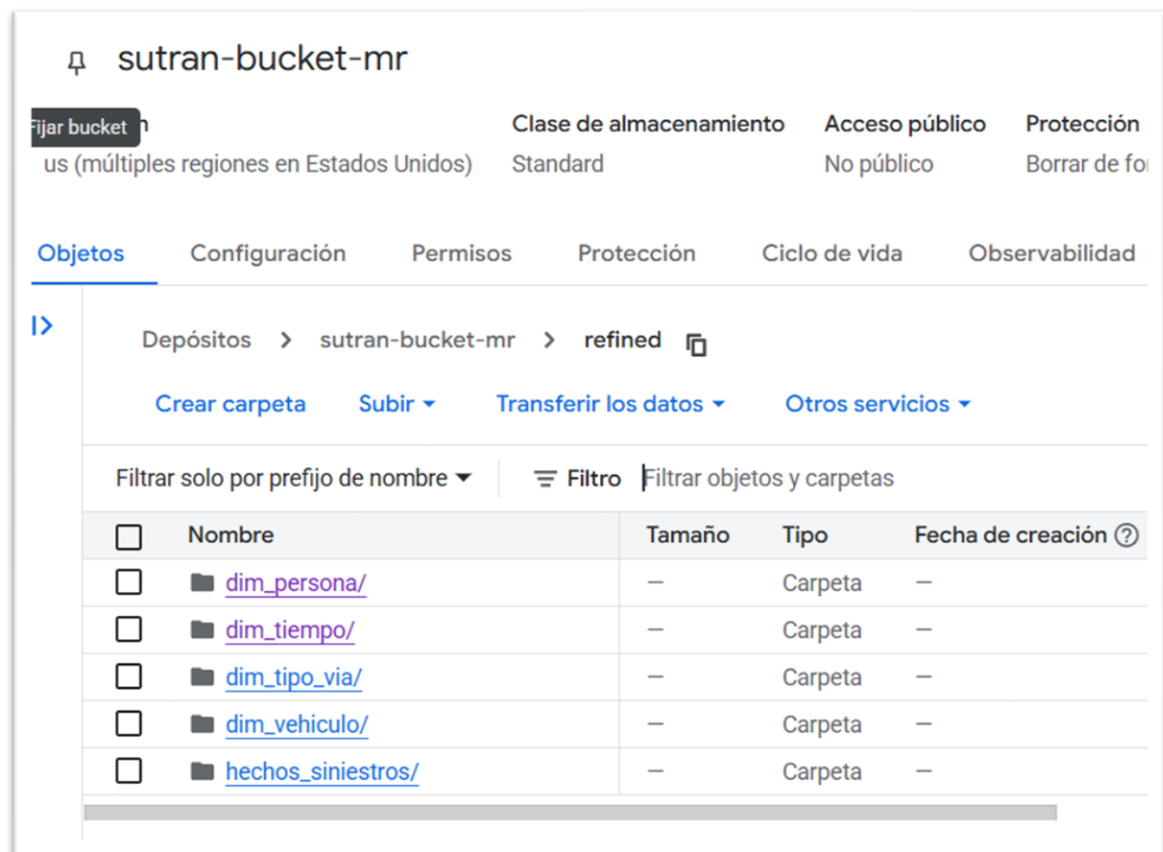
Agregar latitud y longitud del siniestro

```
] df_hechos = df_hechos.join(
    df_siniestros.select(
        "codigo_siniestro",
        col("coordenadas_latitud").alias("latitud"),
        col("coordenadas_longitud").alias("longitud")
    ),
    on="codigo_siniestro",
    how="left"
)
```

Enlazar dimensiones

```
] df_hechos = df_hechos \
    .join(df_dim_tiempo.select("fecha", "id_tiempo"), on="fecha", how="left") \
    .join(df_dim_persona, df_hechos["codigo_persona"] == df_dim_persona["id_persona"], "left") \
    .join(df_dim_vehiculo, df_hechos["codigo_vehiculo"] == df_dim_vehiculo["id_vehiculo"], "left") \
    .join(df_dim_tipo_via.select("tipo_de_via_normalizado", "id_tipo_via"), on="tipo_de_via_normalizado", how="left") \
    .select(
        col("codigo_siniestro").alias("id_siniestro"),
        col("id_persona"),
        col("id_vehiculo"),
        col("id_tiempo"),
        col("id_tipo_via"),
        col("latitud"),
        col("longitud")
    )
```

Construccion_Modelo_Estrella_ONSV.ipynb



Archivos finales en /refined.

4. Modelo dimensional implementado

Para soportar análisis multidimensional de siniestros de tránsito, se implementó un modelo estrella, optimizado para consultas analíticas en BigQuery.

4.1 Tabla de hechos: hechos_siniestros

Grano del hecho:

Una fila por participación asociada a un siniestro (persona + vehículo + tiempo + tipo de vía + ubicación).

Campos principales:

- id_siniestro (STRING) → dimensión degenerada.
- id_persona (STRING).
- id_vehiculo (STRING).
- id_tiempo (INT64).
- id_tipo_via (INT64).
- latitud, longitud.

Métricas derivables:

- Total de registros.
- Total de siniestros.
- Total de personas involucradas.
- Total de vehículos involucrados.

4.2 Dimensiones

a) dim_persona

Permite segmentar siniestros por características de los involucrados.

Atributos clave:

- sexo, edad, gravedad
- estado y clase de licencia
- resultado del dosaje etílico

Uso analítico:

- Análisis de severidad.
- Relación alcohol-siniestros.
- Perfil de riesgo por grupo etario.

b) dim_tiempo

Dimensión temporal con jerarquía completa.

Jerarquía:

- Año → Trimestre → Mes → Día → Fecha

Atributos:

- anio, mes, dia, dia_semana
- trimestre, es_fin_de_semana

Uso analítico:

- Tendencias temporales.
- Comparación entre días hábiles y fines de semana.

c) dim_tipo_via

Atributo principal:

- tipo_de_via_normalizado

Uso analítico:

- Comparación de siniestros por tipo de vía.
- Identificación de zonas de mayor riesgo.

d) dim_vehiculo

Atributos clave:

- modalidad de transporte
- estado SOAT y CITV
- tipo y compañía de seguro
- tipo de vehículo

Uso analítico:

- Análisis normativo.
- Relación entre estado vehicular y siniestros.

4.3 Relaciones del modelo

Relaciones lógicas (no físicas) implementadas en BigQuery:

hechos_siniestros.id_persona → dim_persona.id_persona

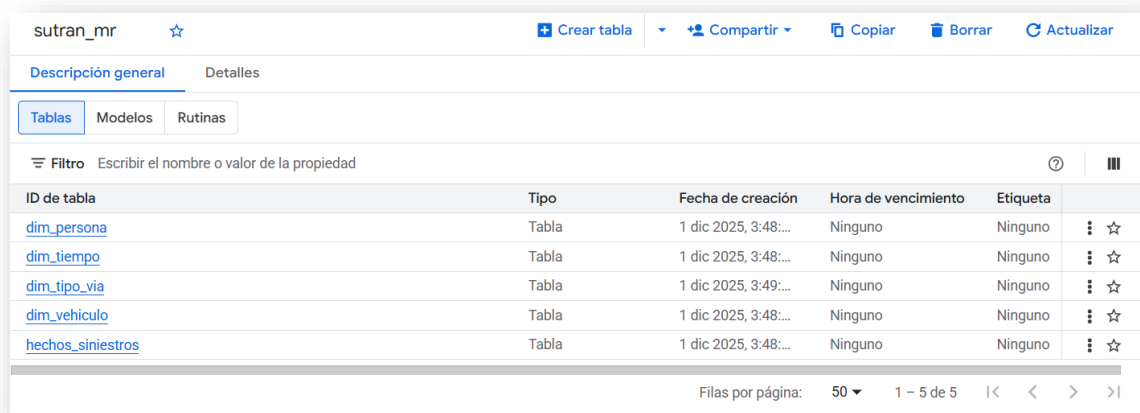
hechos_siniestros.id_vehiculo → dim_vehiculo.id_vehiculo

hechos_siniestros.id_tiempo → dim_tiempo.id_tiempo

hechos_siniestros.id_tipo_via → dim_tipo_via.id_tipo_via

✓ Se optó por FK lógicas para:

- Mejor rendimiento en BigQuery.
- Menor costo computacional.
- Flexibilidad analítica.



ID de tabla	Tipo	Fecha de creación	Hora de vencimiento	Etiqueta	
dim_persona	Tabla	1 dic 2025, 3:48:...	Ninguno	Ninguno	☆
dim_tiempo	Tabla	1 dic 2025, 3:48:...	Ninguno	Ninguno	☆
dim_tipo_via	Tabla	1 dic 2025, 3:49:...	Ninguno	Ninguno	☆
dim_vehiculo	Tabla	1 dic 2025, 3:48:...	Ninguno	Ninguno	☆
hechos_siniestros	Tabla	1 dic 2025, 3:48:...	Ninguno	Ninguno	☆

Tablas creadas en BigQuery (dataset sutran_mr).

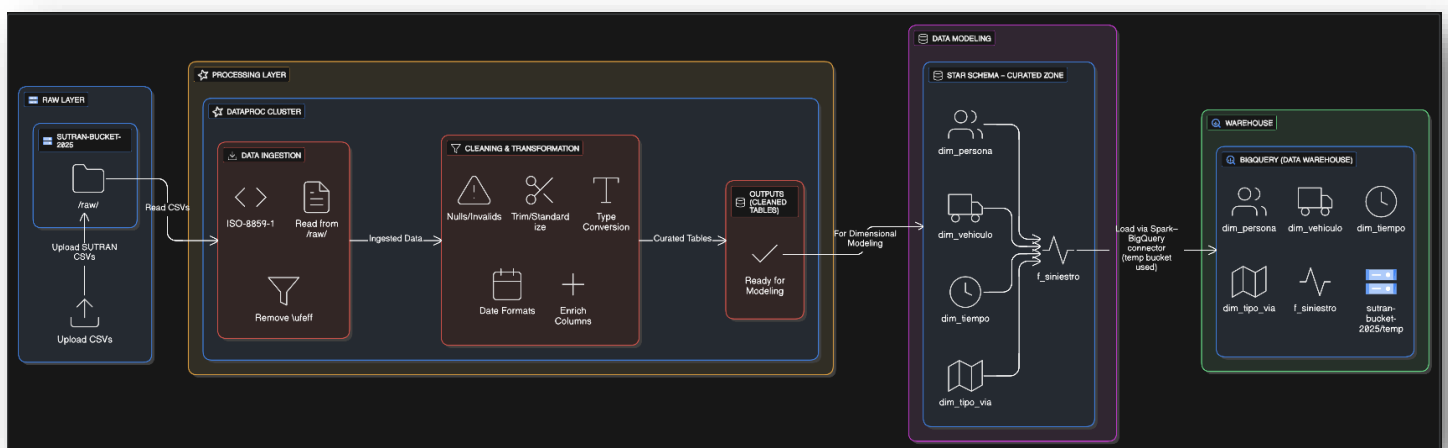
5. Procesos ETL / ELT aplicados

El proyecto implementa un ETL híbrido, combinando procesamiento distribuido en PySpark (ETL) y explotación analítica posterior en BigQuery (ELT analítico). Esta decisión permite separar la carga pesada de transformación del consumo analítico.

5.1 ETL manual inicial (PC3)

En la PC3 se implementó un ETL batch manual utilizando Dataproc + JupyterLab, orientado a validar la lógica de transformación y el modelo dimensional.

Flujo ETL:



1. Extracción

- Lectura de archivos CSV desde /raw en Cloud Storage.

2. Transformación (PySpark)

- Limpieza de codificación y caracteres invisibles.
- Normalización de atributos.
- Conversión de tipos.
- Construcción de dimensiones y tabla de hechos.

3. Carga

- Escritura de datos limpios en /trusted.
- Escritura de datos modelados en /refined.
- Carga final a BigQuery.

```
[1]: project_id = "shaped-icon-478404-p0"
dataset_bq = "sutran_mr"
bucket = "sutran-bucket-mr"
refined_path = f"gs://{bucket}/refined"

spark.conf.set("temporaryGcsBucket", bucket)

def write_to_bq(df, table_name):
    df.write \
        .format("bigquery") \
        .option("table", f"{project_id}:{dataset_bq}.{table_name}") \
        .option("writeMethod", "direct") \
        .mode("overwrite") \
        .save()

# ----- HECHOS -----
df_hechos = spark.read.csv(
    f"{refined_path}/hechos_siniestros/",
    header=True,
    inferSchema=True
)
write_to_bq(df_hechos, "hechos_siniestros")

# ----- DIMENSIONES -----
df_persona = spark.read.csv(f"{refined_path}/dim_persona/", header=True, inferSchema=True)
write_to_bq(df_persona, "dim_persona")

df_vehiculo = spark.read.csv(f"{refined_path}/dim_vehiculo/", header=True, inferSchema=True)
write_to_bq(df_vehiculo, "dim_vehiculo")

df_tiempo = spark.read.csv(f"{refined_path}/dim_tiempo/", header=True, inferSchema=True)
write_to_bq(df_tiempo, "dim_tiempo")

df_tipo_via = spark.read.csv(f"{refined_path}/dim_tipo_via/", header=True, inferSchema=True)
write_to_bq(df_tipo_via, "dim_tipo_via")

print("Carga completada.")
```

Carga completada.

Notebook ejecutándose en Dataproc (JupyterLab).

5.2 ETL automatizado basado en eventos (PC4)

En la PC4, el proceso ETL fue automatizado mediante una arquitectura event-driven, eliminando la intervención manual.

Flujo automatizado:

1. Carga de un nuevo archivo CSV en /raw.
2. Activación automática de una Cloud Function.
3. Ejecución de un job PySpark en Dataproc.
4. Escritura de resultados en /trusted y /refined.
5. Disponibilidad inmediata en BigQuery para análisis.

5.3 Scripts ETL productivos

El procesamiento automatizado se apoya en los siguientes scripts:

- etl_master.py
Script PySpark que implementa la lógica completa del ETL.

etl_master.py

```
import sys
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
```

```

def limpiar_columnas(df):
    for c in df.columns:
        df = df.withColumnRenamed(c, c.strip().lower().replace(" ", "_").replace("á", "a").replace("é",
"e")
                                .replace("í", "i").replace("ó", "o").replace("ú", "u")
                                .replace("ñ", "n"))

    return df

def write_to_bq(df, table_name, project_id, dataset_bq):
    df.write \
        .format("bigquery") \
        .option("table", f"{project_id}.{dataset_bq}.{table_name}") \
        .option("writeMethod", "direct") \
        .mode("overwrite") \
        .save()

if __name__ == "__main__":
    spark = SparkSession.builder.appName("ETL Sutran").getOrCreate()

    # Parámetros de entrada
    input_file = sys.argv[1] # Ej: raw/BDD-ONSV-PERSONAS_2021-2023.csv
    bucket = "sutran-bucket-mr"
    project_id = "shaped-icon-478404-p0"
    dataset_bq = "sutran_mr"

    input_path = f"gs://{bucket}/{input_file}"
    trusted_path = f"gs://{bucket}/trusted/"
    refined_path = f"gs://{bucket}/refined/"

    # Leer archivo
    df = spark.read.option("header", True).option("encoding", "ISO-8859-1").csv(input_path)

    # Tipo de archivo
    if "PERSONAS" in input_file:
        df_clean = df \
            .drop("LUGAR_ATENCION_LESIONADO", "LUGAR_DE_DEFUNCION",
"SE_SOMETIO_A_DOSAJE_ETILICO_CUALITATIVO") \
            .filter(~(col("EDAD") == "No indica")) \
            .withColumn("EDAD", when(col("EDAD") == "No indica", None).otherwise(col("EDAD")))) \
            .dropna(how="all")
        df_clean = limpiar_columnas(df_clean)
        df_clean.write.mode("overwrite").option("header", True).csv(f"{trusted_path}personas/")

    elif "VEHICULOS" in input_file:
        df_clean = df.drop("ELEMENTO_TRANSPORTADO", "AMBITO_SERVICIO").dropna(how="all")
        df_clean = limpiar_columnas(df_clean)
        df_clean.write.mode("overwrite").option("header", True).csv(f"{trusted_path}vehiculos/")

    elif "SINIESTROS" in input_file:
        df_clean = df.drop("EXISTE_SENAL_VERTICAL",
"CLASIFICACION_DE_LA_SENAL_VERTICAL_N_1",
"CLASIFICACION_DE_LA_SENAL_VERTICAL_N_2",
"EXISTE_SENAL_HORIZONTAL") \
            .dropna(how="all")
        df_clean = limpiar_columnas(df_clean)
        df_clean.write.mode("overwrite").option("header", True).csv(f"{trusted_path}siniestros/")

```

```

else:
    print(f"❌ Archivo no reconocido: {input_file}")
    sys.exit(1)

print(f"✅ Limpieza y carga a trusted completada para: {input_file}")

```

- **main.py**

Cloud Function que dispara el job de Dataproc ante eventos en Storage.

```

import os
from google.cloud import dataproc_v1

def trigger_dataproc_job(event, context):
    file = event
    file_name = file['name'] # ej: raw/BBDD_ONSV-PERSONAS_2021-2023.csv
    print(f"Archivo detectado: {file_name}")

    if not file_name.startswith("raw/"):
        print("Archivo fuera de /raw/, se ignora.")
        return

    project_id = os.environ['PROJECT_ID']
    region = os.environ['REGION']
    cluster = os.environ['CLUSTER']

    job_client = dataproc_v1.JobControllerClient(
        client_options={"api_endpoint": f"{region}-dataproc.googleapis.com:443"}
    )

    job = {
        "placement": {"cluster_name": cluster},
        "pyspark_job": {
            "main_python_file_uri": f"gs://sutran-bucket-mr/scripts/etl_master.py",
            "args": [file_name]
        },
    },
}

result = job_client.submit_job(
    project_id=project_id,
    region=region,
    job=job
)

print(f"✅ Job lanzado: {result.reference.job_id}")

```

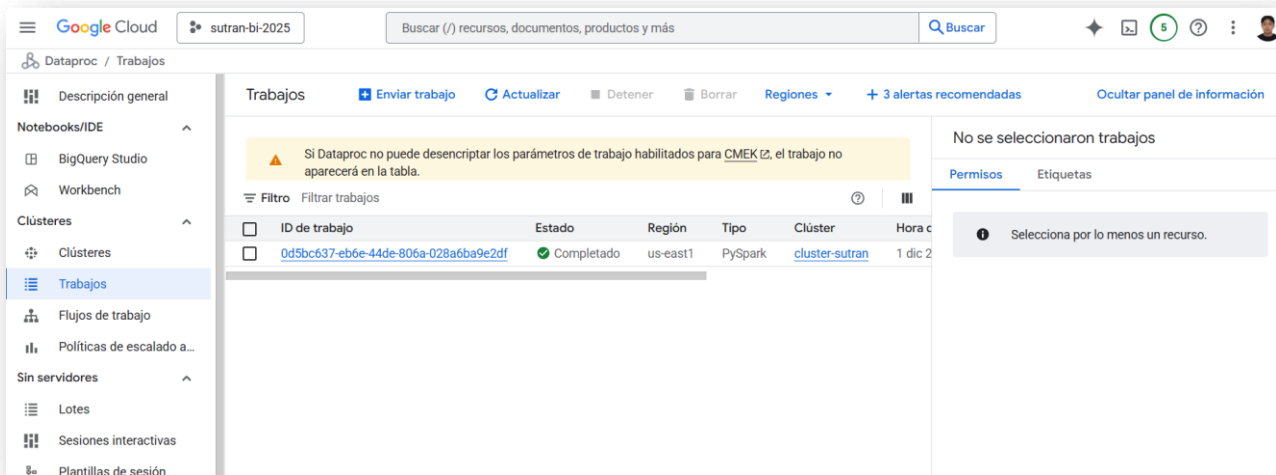
- **requirements.txt**

Dependencias necesarias para la Cloud Function.

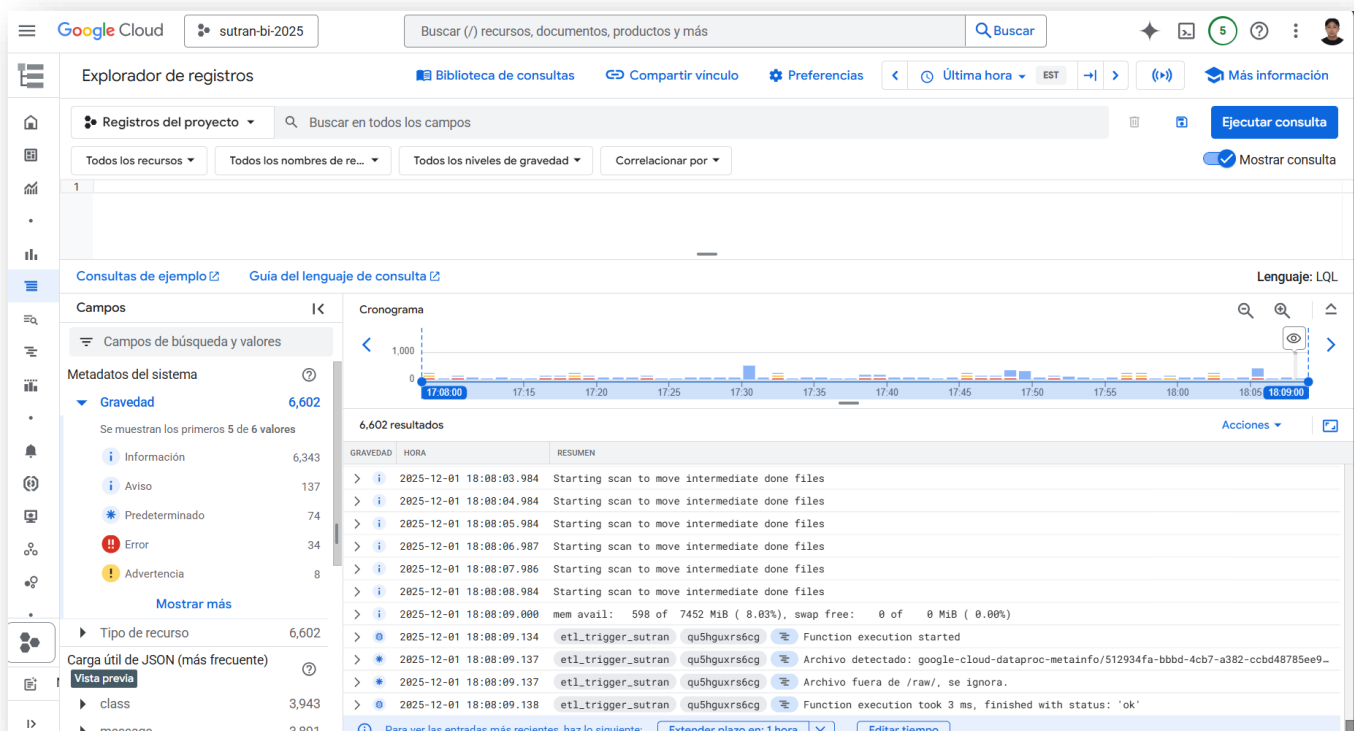
```

google-cloud-storage
google-cloud-dataproc

```

Job Dataproc ejecutándose automáticamente.



Logs visibles en Cloud Logging.

5.4 Justificación técnica ETL / ELT

- PySpark permite procesamiento distribuido y escalable.
- Dataproc reduce la complejidad operativa de Spark.
- BigQuery se utiliza como motor ELT analítico, optimizado para consultas.

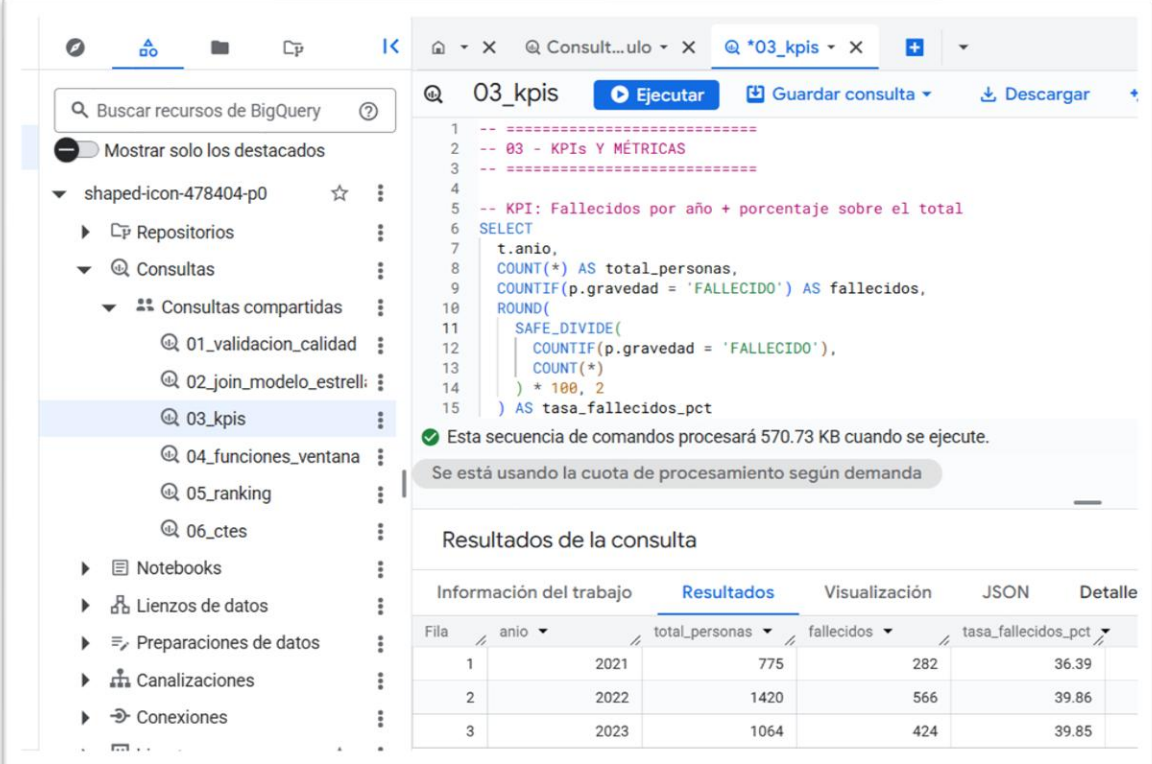
- La automatización por eventos mejora reproducibilidad y confiabilidad.

6. Scripts utilizados y consultas SQL

Para la validación del modelo dimensional y el análisis de siniestros, se desarrollaron **scripts SQL avanzados en BigQuery**, alineados con la rúbrica del curso.

Los scripts se encuentran organizados en:

/grupo10_sutran/scripts/



The screenshot shows the Google Cloud BigQuery interface. On the left, a sidebar lists various resources, including 'Consultas' (Queries) and 'Consultas compartidas' (Shared queries). The '03_kpis' query is selected. The main area displays the SQL query code, which is a SELECT statement calculating the total number of people, the number of deaths, and the death rate percentage for each year. Below the code, a status message indicates that the query will process 570.73 KB. The 'Resultados de la consulta' (Query results) section shows a table with 6 columns: 'Fila' (Row), 'anio' (Year), 'total_personas' (Total people), 'fallecidos' (Deaths), and 'tasa_fallecidos_pct' (Death rate percentage). The results are displayed for the years 2021, 2022, and 2023.

Fila	anio	total_personas	fallecidos	tasa_fallecidos_pct
1	2021	775	282	36.39
2	2022	1420	566	39.86
3	2023	1064	424	39.85

- Estructura de la carpeta scripts/ en GitHub.

```
-- =====
-- 03 - KPIs Y MÉTRICAS
-- =====
-- KPI: Fallecidos por año + porcentaje sobre el total
SELECT
  t.anio,
  COUNT(*) AS total_personas,
  COUNTIF(p.gravedad = 'FALLECIDO') AS fallecidos,
  ROUND(
    SAFE_DIVIDE(
      COUNTIF(p.gravedad = 'FALLECIDO'),
      COUNT(*)
    ) * 100, 2
  ) AS tasa_fallecidos_pct
FROM `sutran.hechos_siniestros` h
```

```

JOIN `sutran.dim_tiempo` t ON h.id_tiempo = t.id_tiempo
JOIN `sutran.dim_persona` p ON h.id_persona = p.id_persona
GROUP BY t.anio
ORDER BY t.anio;

-- KPI: Siniestros por tipo de persona y día de la semana
SELECT
    t.dia_semana,
    p.tipo_persona,
    COUNT(*) AS total
FROM `sutran.hechos_siniestros` h
JOIN `sutran.dim_tiempo` t ON h.id_tiempo = t.id_tiempo
JOIN `sutran.dim_persona` p ON h.id_persona = p.id_persona
GROUP BY t.dia_semana, p.tipo_persona
ORDER BY t.dia_semana, total DESC;

-- KPI: Fines de semana vs días de semana
SELECT
    t.es_fin_de_semana,
    COUNT(*) AS total_personas,
    COUNTIF(p.gravedad = 'FALLECIDO') AS fallecidos,
    ROUND(
        SAFE_DIVIDE(COUNTIF(p.gravedad = 'FALLECIDO'), COUNT(*)) * 100, 2
    ) AS tasa_fallecidos_pct
FROM `sutran.hechos_siniestros` h
JOIN `sutran.dim_tiempo` t ON h.id_tiempo = t.id_tiempo
JOIN `sutran.dim_persona` p ON h.id_persona = p.id_persona
GROUP BY t.es_fin_de_semana;

```

6.1 Validación de datos

Scripts de validación aseguran la calidad del modelo:

- Conteo de registros.
- Detección de duplicados.
- Verificación de valores nulos.
- Consistencia entre hechos y dimensiones.

6.2 Construcción del modelo estrella (SQL)

Se validaron los cruces completos entre la tabla de hechos y las dimensiones mediante consultas SQL.

The screenshot shows the Google Cloud BigQuery console. On the left is a sidebar with a search bar and a list of saved queries and notebooks. The main area displays a SQL query titled '02_join_modelo_estrella'. The query is a complex JOIN operation combining data from 'sutran_hechos_siniestros' with dimension tables 'sutran_dim_persona', 'sutran_dim_vehiculo', 'sutran_dim_tiempo', and 'sutran_dim_tipo_via'. The query includes columns for date, time, location, and vehicle details, and is limited to 200 rows. Below the query, a status message indicates the query size and execution cost. The 'Resultados de la consulta' section shows a table with 7 rows of data, including columns for incident ID, person type, sex, age, severity, and vehicle type.

```

14 t.fecha,
15 t.ano,
16 t.mes,
17 t.dia_semana,
18 t.trimestre,
19 tv.tipo_de_via_normalizado,
20 h.latitud,
21 h.longitud
22 FROM `sutran_hechos_siniestros` h
23 LEFT JOIN `sutran_dim_persona` p ON h.id_persona = p.id_persona
24 LEFT JOIN `sutran_dim_vehiculo` v ON h.id_vehiculo = v.id_vehiculo
25 LEFT JOIN `sutran_dim_tiempo` t ON h.id_tiempo = t.id_tiempo
26 LEFT JOIN `sutran_dim_tipo_via` tv ON h.id_tipo_via = tv.id_tipo_via
27 LIMIT 200;
28

```

Esta consulta procesará 969.45 KB cuando se ejecute.
Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Fila	id_siniestro	tipo_persona	sexo	edad	gravedad	vehiculo
1	A-2021-01-141	PASAJERO	FEMENINO	20	LESIONADO	MOTOCICLE
2	A-2021-01-177	OCUPANTE	FEMENINO	35	LESIONADO	CAMIONETA
3	A-2021-01-177	OCUPANTE	MASCULINO	7	LESIONADO	CAMIONETA
4	A-2021-01-177	OCUPANTE	MASCULINO	13	LESIONADO	CAMIONETA
5	A-2021-01-186	CONDUCTOR FUGADO	MASCULINO	30	ILESO	MOTOCICLE
6	A-2021-01-198	PEATÁIN	MASCULINO	62	FALLECIDO	VEHICULO N
7	A-2021-01-232	PASAJERO	FEMENINO	15	FALLECIDO	MOTOCICLE

Resultados por página: 50 1 - 50 de 200

- 02_join_modelo_estrella.sql

6.3 KPIs y métricas analíticas

Se implementaron scripts para el cálculo de indicadores clave:

- Total de siniestros.
- Distribución por tipo de vía.
- Análisis por gravedad y tiempo.
- Comparaciones temporales.

01_validacion_calidad

Ejecutar

Guardar consulta

```

8 SELECT dim_tipo_via, COUNT(*) FROM `shaped-icon-478404-p0.bi_sutran.di
9 UNION ALL
10 SELECT 'dim_tiempo', COUNT(*) FROM `shaped-icon-478404-p0.bi_sutran.di
11 UNION ALL
12 SELECT 'f_siniestro', COUNT(*) FROM `shaped-icon-478404-p0.bi_sutran.f_
13
14 -- 2) Duplicados en la tabla de hechos por ID de siniestro
15 SELECT id_siniestro, COUNT(*) AS veces
16 FROM `shaped-icon-478404-p0.sutran_mr.hechos_siniestros`
17 GROUP BY id_siniestro
18 HAVING COUNT(*) > 1
19 ORDER BY veces DESC;
20
21 -- 3) Nulos en claves foráneas de la tabla de hechos
22 SELECT
23 SUM(CASE WHEN FECHA IS NULL THEN 1 ELSE 0 END) AS nulos_id_tiempo

```

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo

Resultados

Visualización

JSON

Detail

Fila	id_siniestro	veces
1	A-2023-07-134	33
2	A-2022-02-288	26
3	A-2023-05-136	26
4	A-2023-09-105	21
5	A-2022-09-189	16
6	A-2022-07-119	16
7	A-2022-07-267	16
8	A-2023-09-102	15

- 03_kpis.sql

6.4 Funciones SQL avanzadas

Para análisis temporal y ranking se utilizaron funciones avanzadas:

- Ventanas analíticas (OVER, PARTITION BY).
- Funciones LAG.
- Rankings (RANK).
- CTEs (WITH).

Buscar recursos de BigQuery

Mostrar solo los destacados

- shaped-icon-478404-p0
 - Repositorios
 - Consultas
 - Consultas compartidas
 - 01_validacion_calida
 - 02_join_modelo_estr
 - 03_kpis
 - 04_funciones_ventar
 - 05_ranking
 - 06_ctes
 - Notebooks
 - Lienzos de datos
 - Preparaciones de datos
 - Canalizaciones
 - Conexiones
 - bi_sutran
 - sutran
 - sutran_mr
 - dim_persona

04_funciones_ventana

Ejecutar Guardar consulta Descargar Comp

```

3  -- =====
4
5  -- RANK por tipos de vía según fallecidos
6  SELECT
7    tv.tipo_de_via_normalizado,
8    COUNT(*) AS total_personas,
9    COUNTIF(p.gravedad = 'FALLECIDO') AS total_fallecidos,
10   RANK() OVER (ORDER BY COUNTIF(p.gravedad = 'FALLECIDO') DESC) AS ranking
11 FROM `sutran.hechos_siniestros` h
12 JOIN `sutran.dim_tipo_via` tv ON h.id_tipo_via = tv.id_tipo_via
13 JOIN `sutran.dim_persona` p ON h.id_persona = p.id_persona
14 GROUP BY tv.tipo_de_via_normalizado
15 ORDER BY total_fallecidos DESC;
16
17 -- LAG: evolución mensual de siniestros

```

Esta secuencia de comandos procesará 361.92 KB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo Resultados Visualización JSON Detalles de la ejecución Gráfico

Fila	tipo_de_via_normalizado	total_personas	total_fallecidos	ranking
1	carretera nacional	1724	628	1
2	avenida	726	316	2
3	carretera vecinal	295	117	3
4	carretera departamental	194	76	4
5	carretera sin clasificar	126	46	5
6	jirón	80	37	6
7	calle	71	33	7
8	otro	28	11	8

04_funciones_ventana.sql

-- LAG: evolución mensual de siniestros

```

WITH mensual AS (
  SELECT
    t.anio,
    t.mes,
    COUNT(*) AS total_siniestros
  FROM `sutran.hechos_siniestros` h
  JOIN `sutran.dim_tiempo` t ON h.id_tiempo = t.id_tiempo
  GROUP BY t.anio, t.mes
)
SELECT
  anio,
  mes,
  total_siniestros,
  LAG(total_siniestros) OVER (ORDER BY anio, mes) AS total_mes_anterior,
  total_siniestros - LAG(total_siniestros) OVER (ORDER BY anio, mes) AS
  variacion_abs,
  ROUND(
    SAFE_DIVIDE(
      total_siniestros - LAG(total_siniestros) OVER (ORDER BY anio, mes),
      LAG(total_siniestros) OVER (ORDER BY anio, mes)
    ) * 100, 2
  ) AS variacion_pct
FROM mensual
ORDER BY anio, mes;

```

Fila	anio	mes	total_siniestros	total_mes_anterior	variacion_abs	variacion_pct
2	2021	2	45	22	23	104.55
3	2021	3	66	45	21	46.67
4	2021	4	57	66	-9	-13.64
5	2021	5	52	57	-5	-8.77
6	2021	6	77	52	25	48.08
7	2021	7	83	77	6	7.79
8	2021	8	91	83	8	9.64

- 05_ranking.sql

```
-- Ranking por tipo de persona según fallecidos
SELECT
  p.tipo_persona,
  COUNT(*) AS total_personas,
  COUNTIF(p.gravedad = 'FALLECIDO') AS total_fallecidos,
  RANK() OVER (ORDER BY COUNTIF(p.gravedad = 'FALLECIDO') DESC) AS ranking
FROM `sutran.hechos_siniestros` h
JOIN `sutran.dim_persona` p ON h.id_persona = p.id_persona
GROUP BY p.tipo_persona
ORDER BY total_fallecidos DESC;
```

Resultados de la consulta					
Información del trabajo		Resultados	Visualización	JSON	Detalles de la ejecución
Fila	tipo_persona	total_personas	total_fallecidos	ranking	
1	CONDUCTOR	1351	491	1	
2	PEATÓN	398	382	2	
3	OCUPANTE	575	233	3	
4	PASAJERO	564	165	4	
5	CONDUCTOR FUGADO	335	1	5	
6	SIN CONDUCTOR	36	0	6	

- 06_ctes.sql

7. Visualizaciones finales implementadas

La capa de visualización se implementó utilizando Looker Studio, conectado directamente al Data Warehouse en BigQuery, permitiendo el análisis interactivo de los siniestros de tránsito procesados.

7.1 Objetivo de las visualizaciones

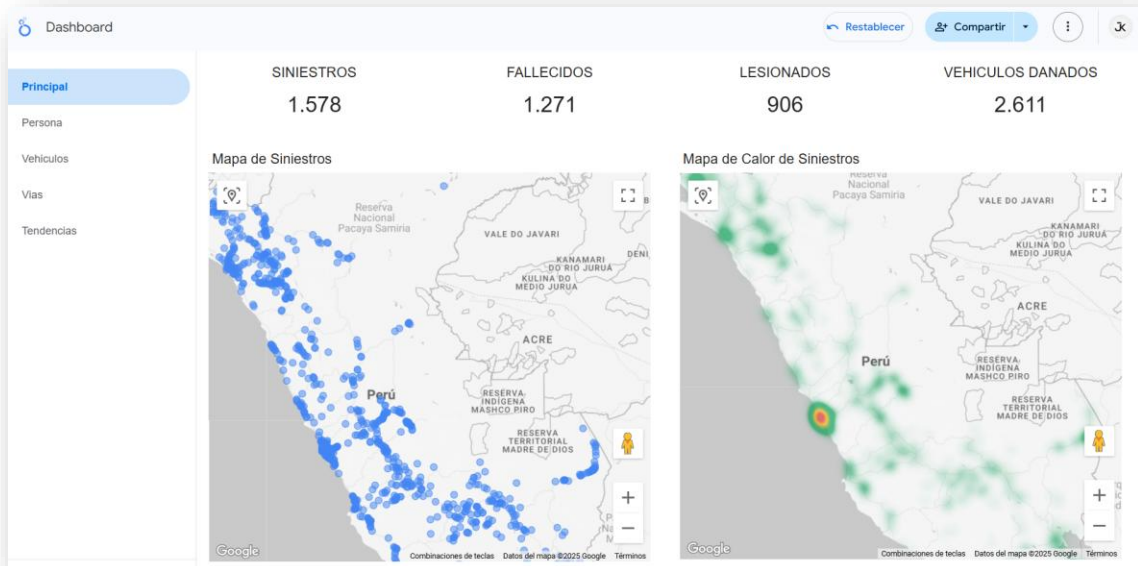
Las visualizaciones tienen como finalidad:

- Facilitar el análisis multidimensional de siniestros.
- Apoyar la toma de decisiones operativas y estratégicas.
- Validar la correcta integración entre ETL, modelo dimensional y capa BI.

7.2 Dashboards implementados

El dashboard principal incluye:

- Indicadores generales de siniestros.
- Análisis temporal (por año, mes, día).
- Segmentación por tipo de vía.
- Análisis por características de personas y vehículos.
- Mapas geográficos basados en latitud y longitud.



- Vista general del dashboard en Looker Studio.
https://lookerstudio.google.com/reporting/3a3d02eb-8e1d-4e2f-bffb-201527848e97/page/p_b1681ecmyd/edit
- Dashboard Looker Studio (enlace público del proyecto).

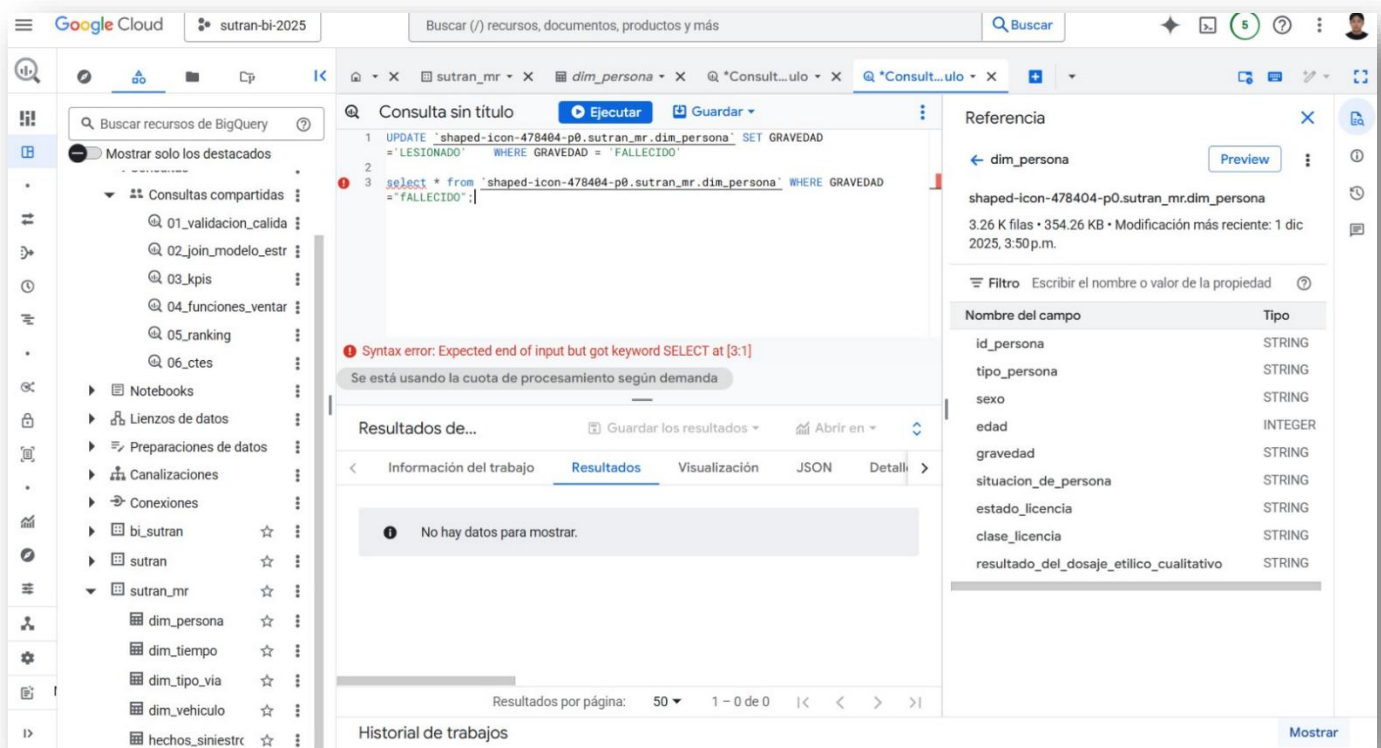
7.3 Validación funcional del dashboard

Para verificar la correcta operación del flujo completo, se realizó una prueba de validación end-to-end:

1. Se modificaron registros en la capa de datos (por ejemplo, cambio de estado de fallecidos a lesionados).
2. Se ejecutó nuevamente el proceso ETL.
3. Se validó que los cambios se reflejaran correctamente en BigQuery.
4. Se confirmó la actualización automática del dashboard.

✓ Esta prueba demuestra:

- Integración correcta entre capas.
- Actualización consistente de datos.
- Confiabilidad del pipeline analítico.





- Evidencia de modificación de datos.
- Evidencia del cambio reflejado en el dashboard.

8. Uso del repositorio GitHub

El proyecto utiliza GitHub como repositorio central para la gestión del código, documentación y evidencias del desarrollo.

8.1 Estructura del repositorio

El repositorio se organiza de forma modular, separando:

- Notebooks ETL.
- Scripts PySpark productivos.
- Scripts SQL analíticos.
- Evidencias (capturas, resultados).
- Documentación del proyecto.

Name	Last commit message	Last commit date
..		
Sesion09	docs	last month
Sesion10	docs: modificacion README.md y bitacora.md agregada	last month
Sesion11	Agregando archivos de evidencia PC4	5 days ago
evidencias	Agregando archivos de evidencia PC4	5 days ago
notebooks	img: pruebas hechas en clase en README	2 weeks ago
scripts	scriptsSQL	2 weeks ago
README.md	Merge branch 'feature/grupo10-jairo-lider' of https://github.com/Si80...	2 weeks ago
bitacora.md	docs: work upload	last month

- Estructura de carpetas del repositorio GitHub.

8.2 Control de versiones

Para asegurar trazabilidad y buenas prácticas de desarrollo:

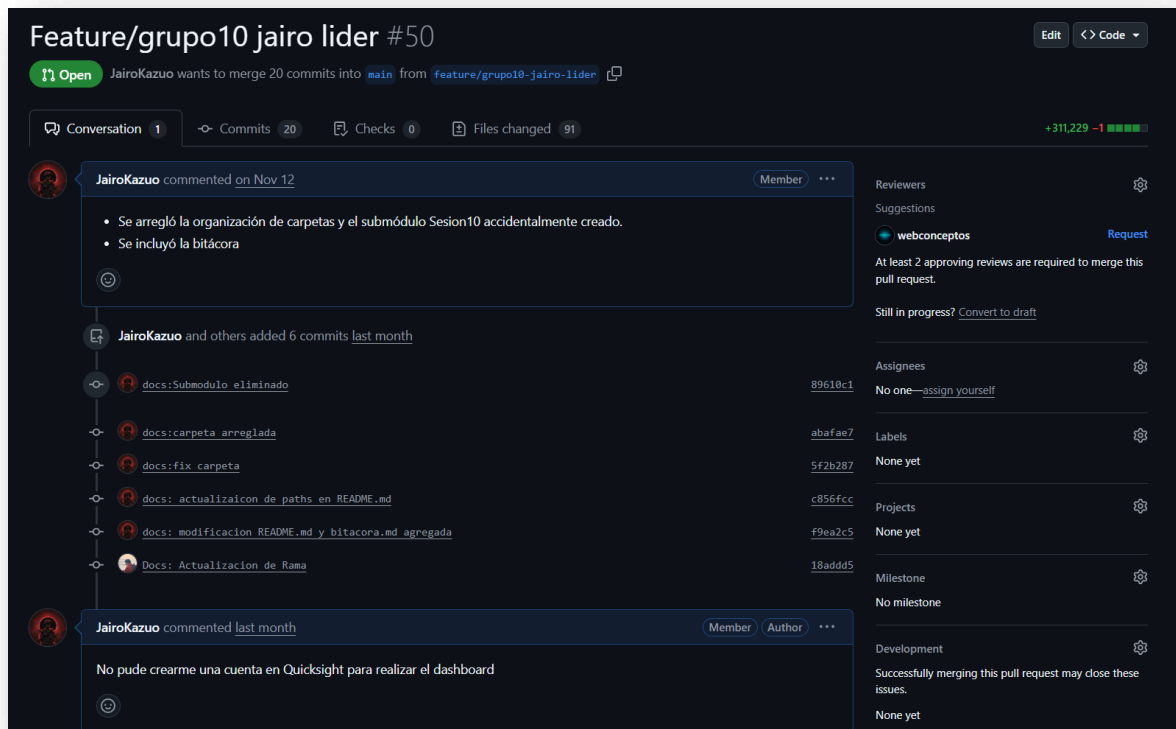
- Se utilizaron ramas por integrante.
- Los cambios fueron integrados mediante Pull Requests (PR).
- Se realizaron merges controlados hacia la rama principal.

✓ Este enfoque permite:

- Historial claro de cambios.
- Revisión del código.
- Evidencia de trabajo colaborativo.



- Lista de ramas del repositorio.
- Pull Requests cerrados.



9. Conclusiones técnicas

A partir del desarrollo del proyecto y las prácticas PC3 y PC4, se concluye que:

- La arquitectura BI implementada en Google Cloud Platform es técnicamente viable para el análisis de siniestros de tránsito.
- El uso del Data Lake Medallion permite trazabilidad, calidad de datos y escalabilidad.
- La combinación de PySpark + Dataproc es adecuada para procesos ETL batch de gran volumen.
- BigQuery ofrece un entorno eficiente para consultas analíticas complejas.
- La automatización basada en eventos mejora la confiabilidad y reproducibilidad del pipeline.
- Las visualizaciones en Looker Studio permiten validar el impacto real del modelo dimensional.
- El uso de GitHub garantiza control de versiones y evidencia del desarrollo técnico.

Como trabajos futuros, recomendamos:

- Incorporar particionado y clustering en BigQuery.
- Migrar a Cloud Functions Gen2 o Cloud Run.
- Implementar Data Catalog y políticas de gobierno de datos.
- Integrar modelos predictivos sobre la capa analítica existente.